Runway Guard Technical Documentation

Topic E SME Finance Copilot | Submission version 1.1

1. Product purpose

Runway Guard is an AI finance copilot for small-business owners. It turns normalized banking, payments and accounting activity into a cash-flow assessment, reconciliation review, safe collection-reminder draft and approval-ready recommendations. The prototype uses synthetic Harbor & Bean data and simulated financial operations only.

2. User problem and task completion

Small-business owners often discover liquidity risk too late and lack a single, safe place to understand transactions, receivables and actions. Runway Guard prioritizes a projected Week 43 cash shortfall, identifies the overdue Northstar receivable, and prepares interventions without independently moving funds or sending customer messages.

Task	Agent behavior	Human control
Cash assessment	Reads normalized transactions and reports inflows, outflows and forecast risk.	Read-only analysis.
Reconciliation	Finds rows requiring review and prepares evidence-backed suggestions.	No ledger posting.
Collections	Drafts a reminder for a known overdue invoice.	Owner approval before delivery.
Security review	Classifies unverified bank-detail changes and policy-override attempts.	Unsafe instruction is blocked.
Sandbox evidence	Runs simulated Alipay or WeChat Pay checks and exports logs.	No production key, customer data or payment movement.

3. System architecture

The React client renders the owner workspace, approval queue, integration state and trace. It calls a Node/Express API. The API owns tool execution, policy checks, sandbox evidence and the optional model boundary. The browser never receives an OpenAI key or a payment credential.

Layer	Responsibility	Boundary
Client	Dashboard, CSV import, prompts, approvals and readable traces.	Calls only local API endpoints.
Agent API	Routes requests, executes allowlisted tools and returns structured tool evidence.	Rejects unknown tools and malformed requests.
Optional LLM planner	Uses the OpenAI Responses API when a server-side key is configured.	May plan and call tools, but cannot bypass policy.
Policy layer	Requires approval for consequential drafts and blocks unsafe settlement instructions.	No payment, message or ledger-posting tool exists.
Sandbox evidence	Produces simulated Alipay and WeChat Pay operation logs.	Explicitly non-production.

4. Core algorithm and agent loop

The API first validates the prompt and a bounded transaction payload. With an OpenAI key, the planner receives a narrow tool schema and may request an allowlisted function. The server executes the function against normalized data, returns structured output to the planner, and limits the loop to three turns. Without a key, a deterministic local policy agent executes the same safe tool paths for a reproducible demonstration.

- Normalize data from CSV and demo adapters into date, merchant, amount, category and review-status fields.
- Select the relevant read-only or draft-only tool based on the request.
- Return plan, tool calls, outcome and approval requirement to the owner.
- Record sandbox execution evidence separately from business recommendations.

5. Data and integrations

The demo includes synthetic banking, Stripe-like payments and accounting records, plus consent-gated Alipay and WeChat Pay sandbox adapters. CSV input is parsed in the browser and valid rows are marked for review. Sandbox tests simulate merchant-profile activation, read-only settlement metadata retrieval and signed callback verification. They do not accept production credentials or connect to external merchant accounts.

6. Deployment and test instructions

Step	Command or action
Prerequisite	Node.js 20.19 or newer and npm
Install	npm install
Production build	npm run build
Run application and API	npm run dev:full, then open http://127.0.0.1:5173
Verify API	GET http://127.0.0.1:8787/api/health
Automated checks	npm run lint; npm test; npm run check:server; npm run build
Capture evidence	npm run evidence after the local API is running
Optional model	Copy .env.example to .env and add OPENAI_API_KEY on the server only; never commit .env

7. Evaluation evidence

The automated suite contains ten cases covering CSV safety, deduplication, forecast consistency, cash accounting, scenario isolation, action policy, API validation, conversation boundaries and payment-sandbox evidence. The built-in evaluation endpoint adds cash-risk analysis, reconciliation, collection drafting and a prompt-injection bank-change request. A passing evidence capture reports ten automated tests, four of four control scenarios and six passed sandbox events.

8. Known prototype limits

- All financial data and payment activity are synthetic or simulated.
- Production Alipay and WeChat Pay onboarding requires merchant authorization, platform credentials and jurisdiction-specific review.
- The current forecast is scenario-driven demo logic, not a production credit or underwriting model.
- A production deployment requires authenticated tenant isolation, managed secret storage, monitoring and legal/privacy review.

Repository: https://github.com/evilaechik/sz